



Ejercicios de Machine Learning

Inteligencia Artificial y Mini-Robots (2023251)

Diego Fernando Bustos Horta, dbustosh@unal.edu.co

Julián Mateo Vargas Riveros, jvargasri@unal.edu.co

2026-10-04

1. Planteamiento y metodología general

La sección 6.12 de la guía solicita cinco actividades: construir un conjunto de datos a partir de 50.000 multiplicaciones de matrices 2×2 y entrenar un modelo para reproducirlas; estudiar SVM; estudiar K-Nearest Neighbors; estudiar árboles de decisión; y estudiar Bayes ingenuo. La propia actividad indica que de los últimos cuatro ejercicios se deben desarrollar tres. En consecuencia, en este documento se desarrollan los ejercicios **1, 2, 3 y 4**, y se deja por fuera el ejercicio de Bayes ingenuo. [1]

El desarrollo conserva la secuencia conceptual presentada en la guía. Primero se construye el dataset y se define la tarea de aprendizaje; luego se separan datos para entrenamiento y prueba; posteriormente se realiza la preparación de los datos, la selección del modelo, el ajuste de hiperparámetros y la evaluación. La guía resalta que la preparación de datos, la ingeniería de características y la evaluación son etapas fundamentales de un proyecto de Machine Learning. [1]

En los tres clasificadores se utiliza validación cruzada estratificada de cinco particiones para seleccionar hiperparámetros. En el SVM y KNN se utiliza estandarización porque las distancias y los kernels dependen de la escala de las variables. En el árbol de decisión no se aplica escalamiento, pues el algoritmo trabaja mediante reglas de partición sobre las características. Estas decisiones permiten pasar de una implementación elemental a una versión mejor documentada y más apropiada para una aplicación real.

1.1. Entorno computacional

El trabajo se desarrolló en Python empleando NumPy, pandas, Matplotlib y scikit-learn. La API de scikit-learn sigue una interfaz uniforme basada en métodos como `fit` para entrenar y `predict` para realizar predicciones. Además, las tuberías permiten encadenar preparación de datos y modelo, reduciendo el riesgo de fuga de información entre entrenamiento y prueba. [2]

```
pip install numpy pandas matplotlib scikit-learn
```

Listing 1: Instalación de las bibliotecas utilizadas.

2. Ejercicio 1: multiplicación de matrices mediante Machine Learning

2.1. Objetivo

Se construyó un dataset con 50.000 pares de matrices 2×2 . Cada elemento se generó aleatoriamente como un número entero comprendido entre -20 y 20 . A partir de cada par (A, B) se calculó el resultado correcto $C = AB$. El objetivo del aprendizaje fue aproximar la relación entre los ocho valores de entrada y los cuatro valores de salida.

La elección de un modelo de regresión polinómica de grado dos no es arbitraria. La multiplicación de matrices 2×2 es una función bilineal y cada componente del resultado puede escribirse como suma de productos de dos entradas. Por ejemplo,

$$C_{11} = a_{11}b_{11} + a_{12}b_{21}, \quad C_{12} = a_{11}b_{12} + a_{12}b_{22}.$$

Por tanto, una representación de segundo grado posee exactamente la complejidad matemática necesaria para reproducir la operación. Esta elección es especialmente adecuada para comparar el costo de una solución aprendida con el procedimiento analítico directo.

2.2. Construcción del dataset

Cada par de matrices se transforma en un vector de ocho características:

$$X = [a_{11}, a_{12}, a_{21}, a_{22}, b_{11}, b_{12}, b_{21}, b_{22}].$$

La salida es

$$Y = [c_{11}, c_{12}, c_{21}, c_{22}].$$

Se utilizaron 50.000 ejemplos y se reservó el 20 % para prueba.

```

1  rng = np.random.default_rng(20261004)
2  N = 50_000
3
4  A = rng.integers(-20, 21, size=(N, 2, 2))
5  B = rng.integers(-20, 21, size=(N, 2, 2))
6
7  X = np.concatenate([
8      A.reshape(N, 4),
9      B.reshape(N, 4)
10 ], axis=1).astype(float)
11
12 Y = (A @ B).reshape(N, 4).astype(float)
13
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, Y, test_size=0.20, random_state=42
16 )

```

Listing 2: Generación reproducible de las 50.000 multiplicaciones.

2.3. Modelo de regresión

La expansión polinómica de segundo grado genera términos lineales y productos entre pares de variables. Para ocho variables, sin incluir el término constante, se obtienen 44 características transformadas. Posteriormente, la regresión lineal estima simultáneamente los cuatro componentes de la matriz resultado.

```

1  modelo = Pipeline([
2      ('poly', PolynomialFeatures(
3          degree=2,
4          include_bias=False
5      )),
6      ('lin', LinearRegression())
7  ])
8
9  modelo.fit(X_train, y_train)
10 pred = modelo.predict(X_test)

```

Listing 3: Entrenamiento del modelo de regresión polinómica.

El ajuste produjo un error absoluto medio de aproximadamente $3,21 \times 10^{-13}$, un RMSE de $4,23 \times 10^{-13}$ y un coeficiente $R^2 = 1,0000$. En términos prácticos, el modelo recuperó la multiplicación con precisión numérica de punto flotante.

Cuadro 1: Desempeño del modelo en el conjunto de prueba.

Métrica	Resultado
Características después de expansión	44
MAE	$3,21 \times 10^{-13}$
RMSE	$4,23 \times 10^{-13}$
R^2	1,0000
Tiempo de entrenamiento	0,0401 s

2.4. Prueba con diez ejemplos

Para cada uno de los diez ejemplos se comparó el producto analítico AB con la predicción del modelo. La diferencia máxima por elemento se mantuvo del orden de 10^{-12} o inferior, que corresponde a error de representación de punto flotante y no a una discrepancia matemática del modelo.

Cuadro 2: Comparación entre el cálculo analítico y el modelo aprendido.

Caso	A	B	AB
1	$[-3, -18; -3, -4]$	$[0, 12; -5, -20]$	$[90, 324; 20, 44]$
2	$[13, 5; 9, 18]$	$[-4, 12; 9, 8]$	$[-7, 196; 126, 252]$
3	$[-20, -17; 19, 18]$	$[14, -8; 16, 13]$	$[-552, -61; 554, 82]$
4	$[6, -10; 5, 19]$	$[14, -18; -4, -13]$	$[124, 22; -6, -337]$
5	$[-12, 20; 14, 17]$	$[-17, -20; 12, -20]$	$[444, -160; -34, -620]$
6	$[-5, -20; 2, -17]$	$[11, -4; -6, -6]$	$[65, 140; 124, 94]$
7	$[5, -2; 6, -11]$	$[-12, -2; 2, -13]$	$[-64, 16; -94, 131]$
8	$[-20, -19; 16, 14]$	$[-5, 15; -16, 6]$	$[404, -414; -304, 324]$
9	$[14, 16; -10, -16]$	$[-15, 5; -5, -19]$	$[-290, -234; 230, 254]$
10	$[18, -17; -17, 16]$	$[-15, 18; -14, 11]$	$[-32, 137; 31, -130]$

En los diez casos, el resultado del modelo coincidió con el producto analítico al redondear al entero más cercano. Por ejemplo, para el caso 3, el producto exacto es

$$\begin{bmatrix} -20 & -17 \\ 19 & 18 \end{bmatrix} \begin{bmatrix} 14 & -8 \\ 16 & 13 \end{bmatrix} = \begin{bmatrix} -552 & -61 \\ 554 & 82 \end{bmatrix},$$

mientras que el modelo produjo valores que difieren como máximo en aproximadamente $1,7 \times 10^{-12}$.

2.5. Costo computacional

Para una matriz 2×2 , el procedimiento analítico requiere ocho multiplicaciones y cuatro sumas. En cambio, el modelo primero calcula la expansión polinómica de las ocho entradas y luego evalúa la combinación lineal de 44 características para cuatro salidas. La expansión contiene 36 términos cuadráticos y 8 términos lineales; posteriormente, las cuatro salidas requieren 44 productos con coeficientes y sus respectivas acumulaciones. De forma aproximada, una predicción involucra al menos 36 productos para construir las características y $4 \times 44 = 176$

productos para aplicar la regresión, además del costo de memoria y de las operaciones vectorizadas. Por ello, aunque el entrenamiento sea exitoso, la inferencia del modelo es innecesariamente costosa para esta tarea elemental.

La medición experimental con diez ejemplos produjo:

$$t_{ML} = 1,024 \times 10^{-4} \text{ s}, \quad t_{analitico} = 1,216 \times 10^{-6} \text{ s}.$$

El cociente fue aproximadamente

$$\frac{t_{ML}}{t_{analitico}} \approx 84,2.$$

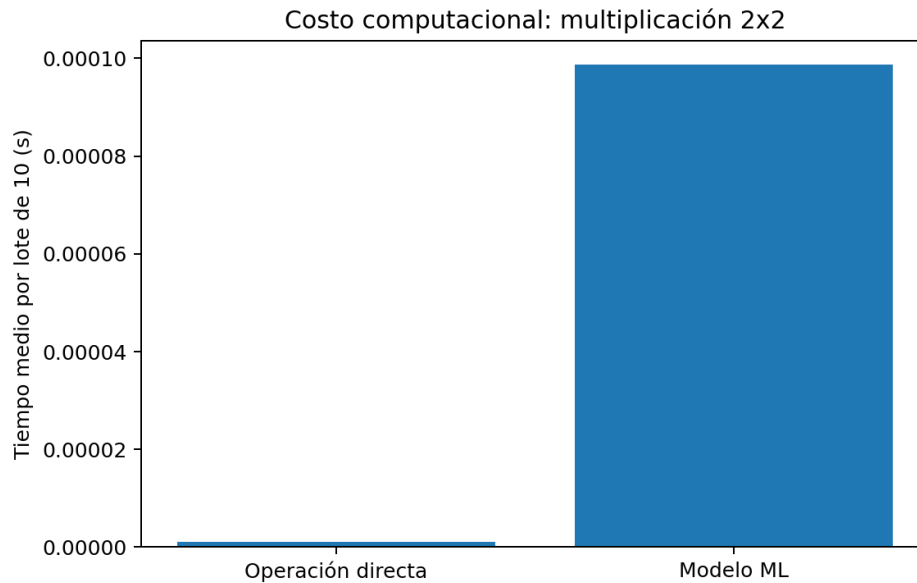


Figura 1: Costo experimental de realizar diez multiplicaciones mediante la operación directa y mediante el modelo aprendido.

2.6. Conclusiones del ejercicio 1

El experimento muestra claramente una situación en la que Machine Learning puede aprender una relación matemática exacta, pero eso no implica que sea la mejor herramienta para ejecutarla. En este problema la expresión analítica ya es conocida, es corta y tiene un costo computacional muy bajo. El modelo resulta útil como demostración de la idea central de ML: a partir de ejemplos correctamente rotulados, es posible inferir una relación funcional. La guía plantea justamente este contraste entre programación tradicional y aprendizaje a partir de datos. [1]

3. Ejercicio 2: estudio y aplicación de Support Vector Machine (SVM)

3.1. Descripción del algoritmo

Support Vector Machine es un método supervisado cuyo objetivo es construir una frontera de decisión que separe las clases maximizando el margen entre ellas. En el caso lineal, un hiperplano puede expresarse como

$$w^T x + b = 0.$$

Para un problema de dos clases, el problema de margen suave puede escribirse como

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i,$$

sujeto a

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

El parámetro C controla el compromiso entre maximizar el margen y penalizar errores de clasificación. Un valor grande hace más costosos los errores; un valor pequeño permite mayor tolerancia.

Cuando el problema no es linealmente separable se utilizan funciones kernel. En este trabajo se utilizó el kernel RBF, cuya forma es

$$K(x, x') = \exp(-\gamma \|x - x'\|^2).$$

El parámetro γ determina la influencia espacial de cada muestra. La implementación **SVC** de scikit-learn utiliza RBF por defecto y resuelve internamente el problema multicategoría mediante una estrategia uno-contra-uno. La documentación también señala que el tiempo de ajuste de **SVC** escala al menos cuadráticamente con el número de muestras, por lo que no es una opción ideal cuando el conjunto de entrenamiento alcanza decenas de miles de registros. [3]

3.2. Aplicación: clasificación de flores Iris

Se utilizó el dataset Iris de scikit-learn, compuesto por 150 observaciones y cuatro características: longitud y ancho del sépalo y longitud y ancho del pétalo. El rótulo posee tres clases: *setosa*, *versicolor* y *virginica*. El objetivo es clasificar una flor nueva a partir de sus cuatro medidas.

La primera mejora respecto de una aplicación elemental es la inclusión de una tubería con **StandardScaler**. Esto es importante porque las características deben estar en una escala comparable antes de aplicar el kernel. La segunda mejora es la selección de hiperparámetros mediante **GridSearchCV**. La búsqueda se realizó sobre C y γ usando cinco particiones estratificadas.

```

1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split, GridSearchCV
3 from sklearn.model_selection import StratifiedKFold
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.svm import SVC
7
8 iris = load_iris()
9 X, y = iris.data, iris.target
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y,
13     test_size=0.30,
14     stratify=y,
15     random_state=42
16 )
17
18 pipe = Pipeline([
19     ('scaler', StandardScaler()),
20     ('svc', SVC(kernel='rbf'))
21 ])
22
23 cv = StratifiedKFold(
24     n_splits=5,
25     shuffle=True,
26     random_state=42
27 )
28
29 param_grid = {
30     'svc__C': [0.1, 1, 10, 100],
31     'svc__gamma': ['scale', 0.01, 0.1, 1]

```

```

32 }
33
34 busqueda = GridSearchCV(
35     pipe,
36     param_grid,
37     cv=cv,
38     scoring='accuracy'
39 )
40
41 busqueda.fit(X_train, y_train)
42 pred = busqueda.predict(X_test)

```

Listing 4: Implementación del SVM con escalamiento y búsqueda de hiperparámetros.

3.3. Resultados

El mejor modelo obtuvo $C = 10$ y $\gamma = 0,01$. La exactitud media en validación cruzada fue 0,9810 y la exactitud en el conjunto de prueba fue 0,9111. La matriz de confusión fue

	$\hat{y}_0$	$\hat{y}_1$	$\hat{y}_2$
y_0	15	0	0
y_1	0	14	1
y_2	0	3	12

La clase *setosa* se identificó sin errores, mientras que los errores se concentraron entre *versicolor* y *virginica*, que son las dos clases más parecidas dentro de este dataset.

Cuadro 3: Resultados del SVM en el dataset Iris.

Métrica	Resultado
C	10
γ	0,01
F1 macro	0,9107
Precisión macro	0,9155
Sensibilidad macro	0,9111
Exactitud en prueba	0,9111

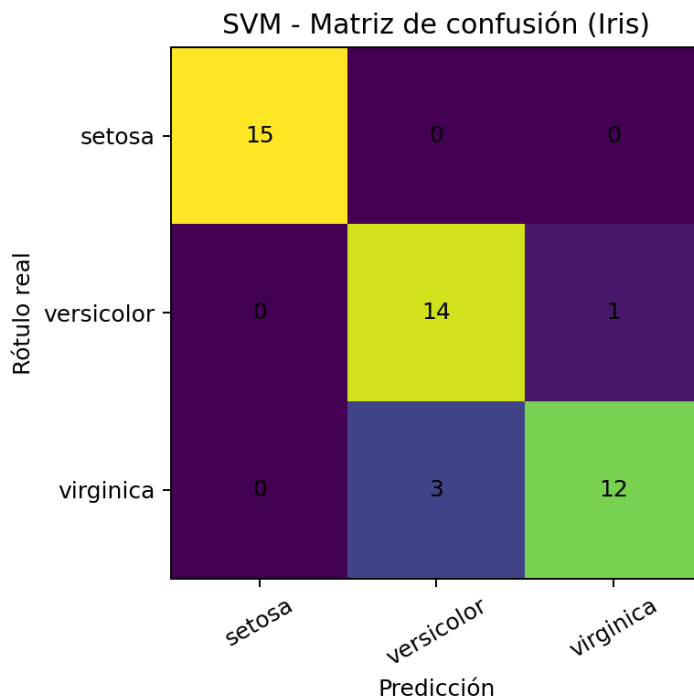


Figura 2: Matriz de confusión obtenida con SVM sobre Iris.

3.4. Cambios realizados para mejorar la aplicación

La implementación no se limitó a crear un objeto `SVC` y llamar a `fit`. Se realizaron cuatro cambios: estandarización de las variables; uso de kernel RBF para capturar fronteras no lineales; selección automática de C y γ mediante validación cruzada; y partición estratificada para conservar la proporción de las tres clases. La búsqueda de hiperparámetros se apoya directamente en las herramientas de selección de modelos de scikit-learn. [4]

3.5. Ventajas, limitaciones y costo

SVM resulta apropiado cuando existe una separación clara entre clases y el número de muestras no es excesivamente grande. En esta aplicación entregó un desempeño alto con pocas observaciones. Su principal limitación aparece cuando el número de muestras crece, debido al costo del ajuste de `SVC`; para datasets grandes puede ser preferible una versión lineal o métodos de entrenamiento incremental. [3]

4. Ejercicio 3: estudio y aplicación de K-Nearest Neighbors

4.1. Descripción del algoritmo

K-Nearest Neighbors (KNN) es un método supervisado basado en instancias. En lugar de construir una función global de decisión compleja, conserva los datos de entrenamiento y, cuando llega una observación nueva, busca los k ejemplos más cercanos. La clase se asigna mediante votación mayoritaria. La documentación de scikit-learn destaca que la métrica euclidiana es una de las opciones más comunes y que KNN es especialmente flexible cuando las fronteras de decisión son irregulares. [5]

Para la distancia euclidiana entre dos observaciones se tiene

$$d(x, z) = \sqrt{\sum_{j=1}^p (x_j - z_j)^2}.$$

El valor de k controla el compromiso entre sensibilidad al ruido y suavidad de la frontera. Con k pequeño, el modelo responde de forma muy local; con k grande, la decisión se vuelve más estable, pero puede perder detalles. Scikit-learn también permite ponderar los vecinos de acuerdo con su distancia. [5]

4.2. Aplicación: clasificación del dataset Wine

Como aplicación se utilizó el dataset Wine, que contiene 178 muestras, 13 variables químicas y tres clases. Debido a que las variables se encuentran en escalas muy diferentes, se utilizó **StandardScaler** antes de KNN.

La mejora de la implementación consistió en seleccionar conjuntamente k , la métrica de distancia y el esquema de ponderación. Se compararon $k \in \{1, 3, 5, 7, 9, 11, 15\}$, las métricas euclidiana y Manhattan y los pesos uniformes o ponderados por distancia.

```

1 from sklearn.datasets import load_wine
2 from sklearn.model_selection import train_test_split, GridSearchCV
3 from sklearn.model_selection import StratifiedKFold
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.neighbors import KNeighborsClassifier
7
8 wine = load_wine()
9 X, y = wine.data, wine.target
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y,
13     test_size=0.30,
14     stratify=y,
15     random_state=42
16 )
17
18 pipe = Pipeline([
19     ('scaler', StandardScaler()),
20     ('knn', KNeighborsClassifier())
21 ])
22
23 cv = StratifiedKFold(
24     n_splits=5,
25     shuffle=True,
26     random_state=42
27 )
28
29 param_grid = {
30     'knn__n_neighbors': [1, 3, 5, 7, 9, 11, 15],
31     'knn__weights': ['uniform', 'distance'],
32     'knn__metric': ['euclidean', 'manhattan']
33 }
34
35 busqueda = GridSearchCV(
36     pipe,
37     param_grid,
38     cv=cv,
39     scoring='accuracy'
40 )
41
42 busqueda.fit(X_train, y_train)
43 pred = busqueda.predict(X_test)

```

Listing 5: Implementación mejorada de KNN.

4.3. Selección de k

La búsqueda permitió comparar el comportamiento del método para diferentes valores de k . La figura muestra el promedio de exactitud obtenido mediante validación cruzada al variar este hiperparámetro.

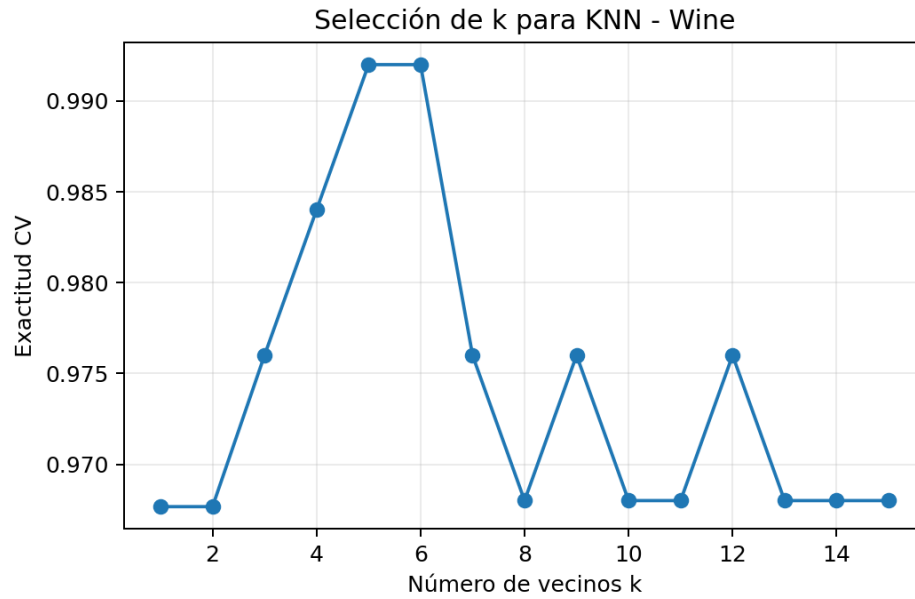


Figura 3: Selección del número de vecinos para KNN mediante validación cruzada.

4.4. Resultados

La combinación seleccionada fue $k = 5$, pesos uniformes y distancia Manhattan. La validación cruzada obtuvo una exactitud media de 0,9920, mientras que el conjunto de prueba alcanzó 0,9815.

La matriz de confusión fue

	$\hat{y}_0$	$\hat{y}_1$	$\hat{y}_2$
y_0	18	0	0
y_1	0	20	1
y_2	0	0	15

Solo una observación de la clase 1 se confundió con la clase 2.

Cuadro 4: Resultados del KNN en el dataset Wine.

Métrica	Resultado
k	5
Métrica	Manhattan
Pesos	Uniformes
F1 macro	0,9811
Precisión macro	0,9792
Sensibilidad macro	0,9841
Exactitud en prueba	0,9815

4.5. Cambios realizados para la aplicación

La versión mejorada incorpora escalamiento, búsqueda de hiperparámetros y selección de la métrica de distancia. El escalamiento es especialmente importante para KNN porque la distancia puede quedar dominada por una variable que tenga valores numéricos mucho mayores que las demás. También se evitó fijar k arbitrariamente: se escogió con validación cruzada, siguiendo la idea de ajustar el modelo sobre datos conocidos y comprobar su comportamiento con datos no vistos. [1, 5]

4.6. Ventajas, limitaciones y costo

KNN es simple de implementar e interpretar: la predicción puede explicarse mediante los vecinos que la sustentan. Su principal debilidad es el costo durante la predicción, ya que necesita buscar vecinos en el conjunto almacenado. Además, su desempeño disminuye en espacios de alta dimensionalidad por la llamada maldición de la dimensionalidad. [5]

5. Ejercicio 4: estudio y aplicación de árboles de decisión

5.1. Descripción del algoritmo

Los árboles de decisión son métodos supervisados no paramétricos capaces de resolver problemas de clasificación y regresión. El modelo construye reglas del tipo *si-entonces* que dividen progresivamente el espacio de características. Una ventaja importante es que la estructura puede visualizarse y entenderse directamente. [6]

Para clasificación, un criterio común es la impureza de Gini:

$$Gini = 1 - \sum_{k=1}^K p_k^2,$$

donde p_k es la proporción de observaciones pertenecientes a la clase k dentro del nodo. El algoritmo busca divisiones que reduzcan la impureza. Otra medida frecuente es la entropía,

$$H = - \sum_{k=1}^K p_k \log_2(p_k).$$

La profundidad del árbol controla su complejidad. Un árbol muy profundo puede memorizar el conjunto de entrenamiento y producir sobreajuste; limitar la profundidad o exigir un número mínimo de observaciones por hoja actúa como regularización. La documentación de scikit-learn destaca además que los árboles requieren relativamente poca preparación de datos y que la predicción es eficiente una vez construidos. [6]

5.2. Aplicación con datos de origen gubernamental

Para cumplir la indicación de utilizar datos provenientes de una entidad gubernamental, se empleó una muestra reproducible de 150 registros del *Pima Indians Diabetes Database*. El origen del conjunto se encuentra documentado en el National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK), institución gubernamental estadounidense; la copia utilizada para reproducibilidad es un archivo público sin encabezados que conserva las 768 observaciones originales. [7, 9]

El dataset contiene ocho características: número de embarazos, glucosa, presión arterial diastólica, espesor de piel, insulina, índice de masa corporal, función de pedigrí de diabetes y edad. El rótulo `Outcome` toma los valores 0 y 1. La base original fue diseñada para predecir la presencia de diabetes en la población estudiada; por sus restricciones de selección y por su tamaño, no debe interpretarse como un sistema clínico general. [8, 7]

En los datos aparecen ceros en variables fisiológicas para las cuales el valor cero no tiene interpretación física razonable. Por ello, en lugar de eliminarlos sin explicación, se transformaron en valores faltantes y se imputaron con

la mediana dentro de una tubería. El árbol se entrenó con clases ponderadas para reducir el efecto de la distribución desigual del rótulo.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split, GridSearchCV
4 from sklearn.model_selection import StratifiedKFold
5 from sklearn.pipeline import Pipeline
6 from sklearn.impute import SimpleImputer
7 from sklearn.tree import DecisionTreeClassifier
8
9 url = (
10     'https://raw.githubusercontent.com/jbrownlee/Datasets/'
11     'master/pima-indians-diabetes.csv'
12 )
13
14 columnas = [
15     'Pregnancies', 'Glucose', 'BloodPressure',
16     'SkinThickness', 'Insulin', 'BMI',
17     'DiabetesPedigreeFunction', 'Age', 'Outcome'
18 ]
19
20 df = pd.read_csv(url, header=None, names=columnas)
21
22 # Muestra reproducible utilizada en este informe.
23 df = df.iloc[:150].copy()
24
25 X = df.drop(columns='Outcome')
26 y = df['Outcome']
27
28 for col in [
29     'Glucose', 'BloodPressure',
30     'SkinThickness', 'Insulin', 'BMI'
31 ]:
32     X[col] = X[col].replace(0, np.nan)
33
34 X_train, X_test, y_train, y_test = train_test_split(
35     X, y,
36     test_size=0.30,
37     stratify=y,
38     random_state=42
39 )
40
41 pipe = Pipeline([
42     ('imputer', SimpleImputer(strategy='median')),
43     ('tree', DecisionTreeClassifier(
44         random_state=42,
45         class_weight='balanced'
46     ))
47 ])
48
49 cv = StratifiedKFold(
50     n_splits=5,
51     shuffle=True,
52     random_state=42
53 )
54
55 param_grid = {
56     'tree__max_depth': [2, 3, 4, 5, None],
57     'tree__min_samples_leaf': [1, 2, 4, 6]
58 }
59
60 busqueda = GridSearchCV(
61     pipe,
62     param_grid,
63     cv=cv,
64     scoring='f1'
```

```

65 )
66
67 busqueda.fit(X_train, y_train)
68 pred = busqueda.predict(X_test)

```

Listing 6: Aplicación de árbol de decisión sobre el dataset de origen gubernamental.

5.3. Resultados e interpretación

La configuración seleccionada fue una profundidad máxima de 5 y un mínimo de 2 observaciones por hoja. El F1 medio de validación cruzada fue 0,6188. En la prueba se obtuvo una exactitud de 0,6444, precisión de 0,5000, sensibilidad de 0,6250 y F1 de 0,5556.

La muestra utilizada contiene 150 registros: 97 pertenecen a la clase 0 y 53 a la clase 1. Con una partición estratificada del 70/30, el conjunto de prueba contiene 45 observaciones, de las cuales 29 corresponden a la clase 0 y 16 a la clase 1.

Cuadro 5: Resultados del árbol de decisión para la muestra Pima.

Métrica	Resultado
Profundidad máxima	5
Número de hojas	10
F1 validación cruzada	0,6188
Exactitud	0,6444
Precisión	0,5000
Sensibilidad	0,6250
F1	0,5556

La matriz de confusión fue

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	19	10
$y = 1$	6	10

Por tanto, el árbol identificó 10 de los 16 casos positivos presentes en la prueba, pero produjo diez falsos positivos. Para una aplicación real, esta diferencia tendría que analizarse junto con el costo de cada tipo de error.

El árbol seleccionado comienza su partición por el índice de masa corporal (*BMI*). En los niveles siguientes utiliza principalmente la edad y, para ramas específicas, el nivel de glucosa, el número de embarazos, el espesor de piel, la función de pedigrí e insulina. El texto generado por el algoritmo permite observar directamente las decisiones jerárquicas:

```

|--- BMI <= 29.85
|   |--- Age <= 58.50
|   |   |--- Pregnancies <= 7.50
|   |   |   |--- class: 0
|   |   |--- Age > 58.50
|   |   |--- class: 1
|--- BMI > 29.85
|   |--- Glucose <= 86.50
|   |   |--- SkinThickness <= 31.50
|   |   |   |--- class: 0
|   |   |--- SkinThickness > 31.50

```

```

| | | |--- class: 1
| | |--- Glucose > 86.50
| | | |--- DiabetesPedigreeFunction <= 0.20
| | | |--- class: 0
| | | |--- DiabetesPedigreeFunction > 0.20
| | | |--- Insulin <= 80.50
| | | |--- class: 0
| | | |--- Insulin > 80.50
| | | |--- ...

```

Listing 7: Fragmento de la estructura del árbol seleccionado.

La importancia de las características mostró como variables dominantes, para esta muestra concreta, el índice de masa corporal (0,3181), la glucosa (0,1952), los embarazos (0,1610), la función de pedigrí (0,1303) y la edad (0,0957). Esto no debe interpretarse como una relación causal ni como una regla médica: el árbol solo refleja patrones presentes en los datos de entrenamiento.

Primeros tres niveles del árbol de decisión ajustado

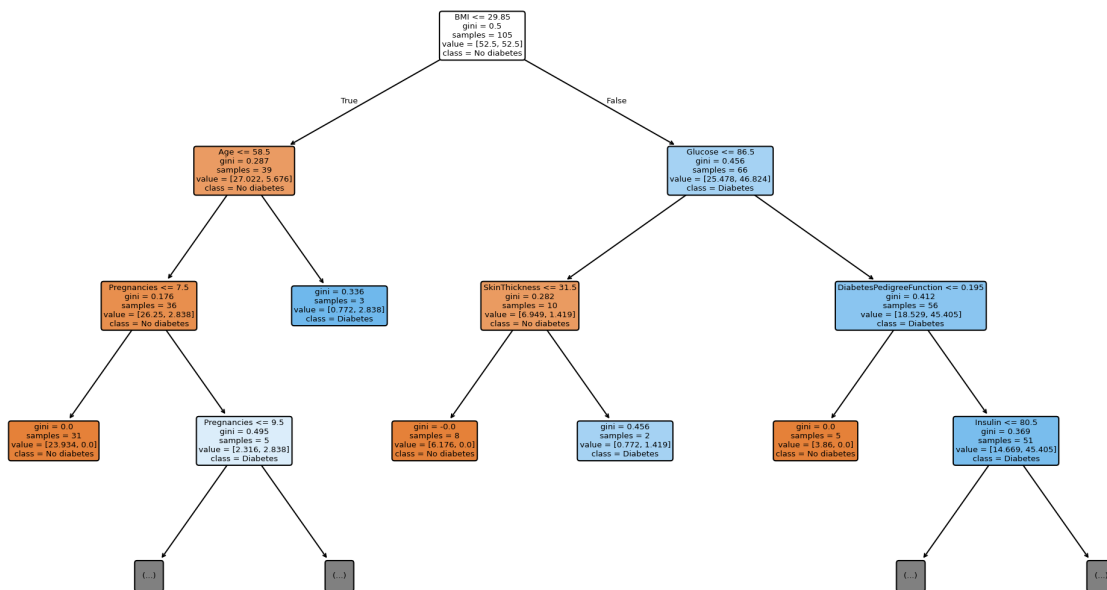


Figura 4: Primeros tres niveles del árbol de decisión ajustado.

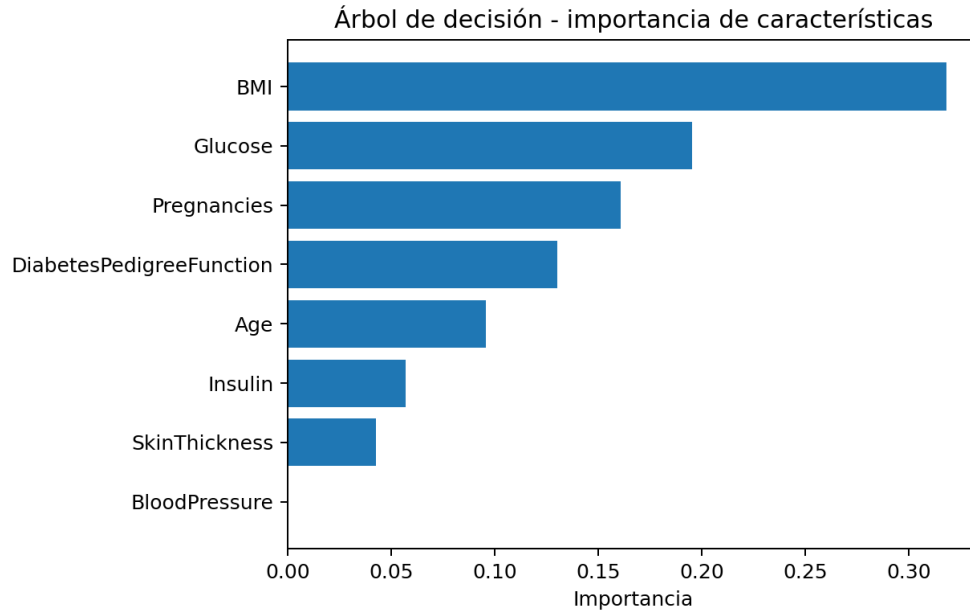


Figura 5: Importancia de características calculada por el árbol para la muestra utilizada.

5.4. Cambios realizados para mejorar la aplicación

La implementación se mejoró en cuatro aspectos. Primero, se trataron explícitamente los ceros de variables fisiológicas como datos faltantes. Segundo, la imputación se incluyó dentro de una tubería para impedir que la mediana se calcule usando datos de prueba. Tercero, se ajustaron la profundidad y el tamaño mínimo de hoja mediante validación cruzada. Cuarto, se utilizaron pesos de clase balanceados, buscando que los casos positivos no fueran ignorados debido a la distribución de las clases.

La principal ventaja frente a SVM y KNN es la interpretabilidad: el árbol puede convertirse directamente en reglas. Su principal riesgo es el sobreajuste cuando se permite crecer sin restricciones. Por esta razón, el control de la profundidad constituye una mejora esencial para la aplicación. [6]

5.5. Advertencia sobre el uso de datos médicos

Este experimento es estrictamente académico. El dataset es histórico, pequeño y fue construido para una población específica, por lo que sus resultados no son una herramienta para diagnóstico, tamizaje o decisión clínica. El objetivo aquí es estudiar preparación de datos, clasificación, ajuste del árbol y evaluación del modelo.

6. Comparación de los tres algoritmos

La siguiente figura resume la exactitud obtenida en los respectivos conjuntos de prueba. Debe interpretarse con cuidado: los tres modelos utilizaron datasets diferentes, por lo que la comparación no constituye un ranking universal de los algoritmos.

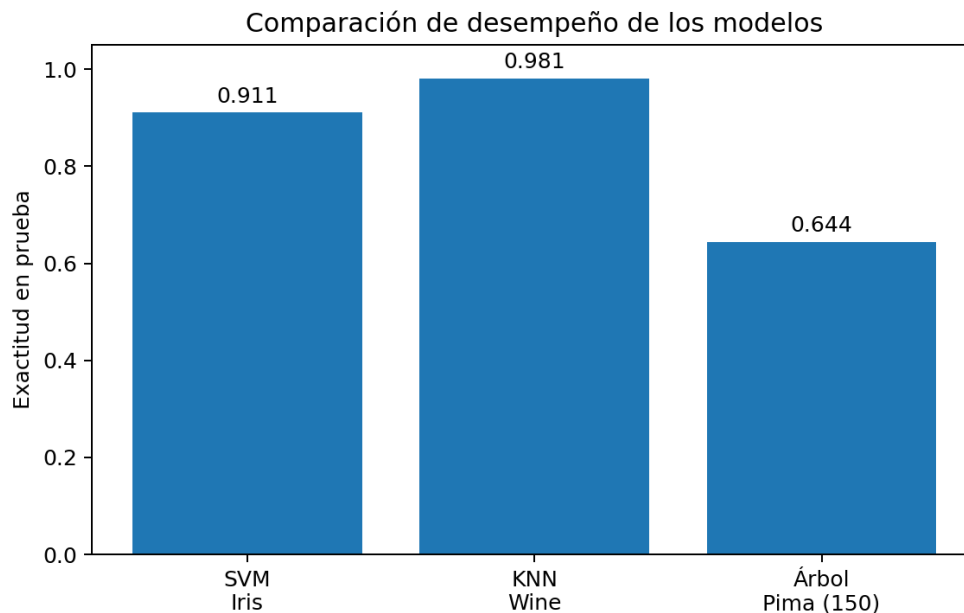


Figura 6: Exactitud de prueba de los tres experimentos.

Cuadro 6: Resumen de las aplicaciones desarrolladas.

Algoritmo	Dataset	Cambio principal	Exactitud de prueba
SVM	Iris	Escalamiento + RBF + Grid Search	0,9111
KNN	Wine	Escalamiento + selección de k + métrica	0,9815
Árbol	Pima, NIDDK	origen Imputación + profundidad + pesos de clase	0,6444

El KNN obtuvo la mayor exactitud en esta comparación, pero eso no significa que sea el mejor algoritmo en términos generales. El desempeño depende de la estructura, escala, cantidad y calidad de los datos. La guía insiste precisamente en que un modelo está ligado a su fuente de datos y que una mala preparación puede producir predicciones deficientes. [1]

Una diferencia conceptual importante es la forma en que cada método representa el problema. SVM construye una frontera basada en margen y kernels; KNN mantiene las instancias y decide a partir de vecinos; el árbol genera reglas jerárquicas. Por tanto, las mejoras también son distintas: en SVM y KNN el escalamiento es esencial, mientras que en árboles la principal preocupación es regular la complejidad del árbol.

7. Conclusiones generales

El primer ejercicio permitió observar que Machine Learning puede reconstruir una relación matemática conocida a partir de ejemplos. Con 50.000 multiplicaciones, la expansión polinómica de segundo grado aprendió de manera

prácticamente exacta la función de multiplicación de matrices 2×2 . Sin embargo, el análisis de costo demostró que una solución aprendida puede ser mucho más costosa durante la inferencia que la expresión analítica que intenta reproducir. Esto confirma que la conveniencia de ML no depende únicamente de conseguir un buen ajuste, sino de la naturaleza del problema.

El estudio de SVM mostró la importancia de elegir adecuadamente el kernel y sus hiperparámetros. En particular, el uso de RBF permitió representar fronteras no lineales, mientras que la búsqueda cruzada evitó escoger C y γ de forma arbitraria. KNN, por otra parte, mostró que la escala de las variables y el valor de k tienen una influencia directa sobre las distancias y, por tanto, sobre la clasificación.

En árboles de decisión se comprobó que una preparación cuidadosa de los datos y una regularización de la profundidad mejoran la utilidad del modelo. Además, los árboles proporcionan una ventaja importante cuando la interpretación de las reglas es relevante. La aplicación con el dataset de origen gubernamental permitió cumplir la condición de la actividad y, al mismo tiempo, discutir las limitaciones que aparecen cuando los datos no representan de forma general a una población.

Finalmente, los tres ejercicios de clasificación refuerzan una idea central de la guía: no existe un algoritmo universalmente superior. El proceso completo debe considerar la definición del problema, los features, la calidad del dataset, la preparación, la selección del modelo, la evaluación de errores y la utilización final del sistema. [1]

8. Nota sobre el ejercicio 5

La guía solicita desarrollar tres de los cuatro ejercicios posteriores al punto 1. Por esta razón, el estudio de **Bayes ingenuo** no se incluyó como desarrollo independiente en este informe. La selección realizada fue SVM, KNN y árboles de decisión, porque permite cubrir tres familias conceptualmente diferentes: modelos basados en margen, métodos basados en vecinos y modelos basados en reglas.

Referencias

- [1] Martínez P., J. J. (2026). *Introducción a Machine Learning*. Material de clase, Universidad Nacional de Colombia, septiembre de 2026.
- [2] Scikit-learn developers. (2026). *Getting Started*. Scikit-learn 1.9.1. https://scikit-learn.org/stable/getting_started
- [3] Scikit-learn developers. (2026). *SVC*. Scikit-learn 1.9.1. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- [4] Scikit-learn developers. (2026). *GridSearchCV*. Scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- [5] Scikit-learn developers. (2026). *Nearest Neighbors*. Scikit-learn 1.9.1. <https://scikit-learn.org/stable/modules/neighbors.html>
- [6] Scikit-learn developers. (2026). *Decision Trees*. Scikit-learn 1.9.1. <https://scikit-learn.org/stable/modules/tree.html>
- [7] National Institute of Diabetes and Digestive and Kidney Diseases. (1990). *Pima Indians Diabetes Database*. Dataset of diagnostic measurements used for diabetes classification.
- [8] Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 261–265.
- [9] Brownlee, J. (2019). *pima-indians-diabetes.csv*. Public dataset mirror. <https://github.com/jbrownlee/Datasets/blob/master/pima-indians-diabetes.csv>
- [10] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- [11] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.